Password Guessing Based on Deep Learning

Tianrui Gu
Duke Kunshan University
Yuan Pei Young Scholars Program, Peking University
Advisor: Prof. Lianghuan Kang, Peking University

August 2024

Abstract

Passwords remain the most widely deployed authentication mechanism, yet users routinely choose weak, reused, or predictable credentials. Estimating password strength therefore requires realistic guessing attacks. This paper studies a generative adversarial network (GAN) approach to password guessing. Building on PassGAN and WGAN-GP, we modify the generator by adding a linear block and inserting batch normalization before each ReLU in the residual blocks. Using the RockYou password corpus, we compare the proposed model with Markov models, HashCat, FLA, and the original PassGAN. The modified model achieves higher hit rates than PassGAN across the evaluated baselines. We further analyze how password length and the ratio of letters to digits affect guessing success, and use these findings to derive practical recommendations for users.

Keywords: password guessing; generative adversarial networks; WGAN-GP; PassGAN; information security.

1 Introduction

Passwords are popular because they are easy to deploy and easy to use, but they are also vulnerable to guessing attacks. Large-scale password leaks have repeatedly shown that users choose predictable patterns: common words, names, dates, keyboard sequences, and simple character substitutions. Since password strength is a statistical property reflecting resistance to guessing, a reliable estimate of strength requires simulating realistic guessing attacks rather than relying only on composition rules.

Password leakage is a persistent security problem. Users often reuse passwords across services, embed personal information, or apply predictable substitutions; surveys report that a substantial fraction of users reuse passwords across platforms [4]. These habits make passwords vulnerable even when individual passwords appear complex.

Traditional guessing methods are largely dictionary-based or rule-based. Tools such as John the Ripper enumerate candidate passwords from dictionaries and transformation rules [26]. Statistical approaches model password generation as a stochastic process: Narayanan and Shmatikov introduced a time–space tradeoff for dictionary attacks [3], and later work used n-gram Markov models, probabilistic context-free grammars, and ordered enumeration [28, 29]. These methods are efficient and interpretable, but they depend on hand-designed features and can overfit the training corpus.

Deep learning offers a different approach: a model can learn the distribution of passwords directly from leaked data and generate candidates that are not limited to explicit rules. Recurrent networks and generative adversarial networks have both been applied to this problem [30, 31, 32, 1]. PassGAN, in particular, treats

password generation as a distribution-matching problem and uses a Wasserstein GAN with gradient penalty (WGAN-GP). Its ability to generate realistic passwords attracted attention from security researchers and the media [6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20].

This paper follows that line of work and makes three contributions. First, we adapt a PassGAN-style model with WGAN-GP and a modified residual generator. Second, we compare its hit rate with Markov models, HashCat, FLA, and the original PassGAN on the RockYou corpus. Third, we analyze how password length and character composition affect guessing success and translate the results into user-facing security recommendations.

2 Related Work

2.1 Password Guessing

Dictionary attacks generate candidates from word lists and mangling rules. John the Ripper is a widely used example; its effectiveness depends on whether the target password or a close variant appears in the dictionary [26]. Markov-based methods model the conditional probability of a character given the previous n characters [3]. Higher-order models capture more structure but increase the parameter space and the risk of overfitting; smoothing and ordered enumeration were introduced to address these issues [28, 29].

Probabilistic models assign a likelihood to candidate passwords and enumerate them in decreasing order. Ma et al. studied probabilistic password models and smoothing for sparse n-grams [28]; Tansey proposed improved models for password guessing [27]; OMEN orders Markov-model candidates to improve guessing efficiency [29]. These methods provide strong baselines, which we include in our evaluation.

Deep-learning methods learn a generative model of passwords from data. Melicher et al. proposed FLA, a recurrent model for fast password-strength estimation [30]. Xu et al. used LSTM networks to capture long-range dependencies [31], and Xia and Yi combined probabilistic context-free grammars with recurrent networks [32]. Hitaj et al. introduced PassGAN, which applies WGAN-GP with one-dimensional convolutions to character-level password generation [1]. Our work builds directly on PassGAN.

PassGAN is built on WGAN-GP, which itself evolved from the original GAN through WGAN. Related GAN variants include DCGAN, adversarial autoencoders, and loss-sensitive GANs [34, 25, 35, 24, 23]. The key difficulty addressed by WGAN-GP is the instability of the original minimax objective when the real and generated distributions have disjoint supports.

2.2 Generative Adversarial Networks

A GAN consists of a generator G and a discriminator D trained in opposition [34]. The generator maps a latent noise vector z to a synthetic sample $G(z)$; the discriminator estimates whether a sample comes from the real distribution p_r or the generated distribution p_g. The minimax objective is

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_r}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]. \tag{1}$$

For a fixed generator, the optimal discriminator is

$$D^*(x) = \frac{p_r(x)}{p_r(x) + p_g(x)}. \tag{2}$$

Substituting (2) into (1) shows that the objective is related to the Jensen–Shannon (JS) divergence:

$$V(D^*, G) = 2\,\mathrm{JS}(p_r \| p_g) - 2\log 2. \tag{3}$$

The Kullback–Leibler (KL) and Jensen–Shannon divergences are defined as

$$\mathrm{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}, \qquad \mathrm{JS}(p\|q) = \frac{1}{2}\mathrm{KL}(p\|m) + \frac{1}{2}\mathrm{KL}(q\|m), \tag{4}$$

where $m = (p + q)/2$. When the two distributions have disjoint supports, the JS divergence saturates at $\log 2$, and the gradient with respect to the generator vanishes. This is one explanation for the training instability of the original GAN.

3 Methodology

3.1 Wasserstein GAN

WGAN replaces the JS divergence with the Wasserstein-1 distance [25]:

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y)\sim\gamma} \left[\|x - y\|\right]. \tag{5}$$

Using the Kantorovich–Rubinstein duality, this can be written as

$$W(p_r, p_g) = \sup_{\|f\|_L \le 1} \Big(\mathbb{E}_{x\sim p_r}[f(x)] - \mathbb{E}_{x\sim p_g}[f(x)] \Big), \tag{6}$$

where $\|f\|_L \le 1$ denotes the 1-Lipschitz constraint. Formally, a function f is K-Lipschitz if

$$|f(x_1) - f(x_2)| \le K\|x_1 - x_2\| \qquad \text{for all } x_1, x_2. \tag{7}$$

In WGAN, the discriminator (called the critic) is trained to approximate f, and the constraint is enforced by weight clipping. The clipping threshold controls the Lipschitz constant: a threshold that is too small can make gradients vanish, while a threshold that is too large can make them explode. Weight clipping therefore restricts the critic to a small subset of Lipschitz functions and can produce unstable training. WGAN also mitigates mode collapse, in which the generator produces a narrow set of samples, but its main benefit is training stability rather than a direct improvement in sample quality [25].

3.2 WGAN-GP

WGAN-GP replaces weight clipping with a gradient penalty [35]. The critic loss is

$$\mathcal{L}_D = \mathbb{E}_{\tilde{x}\sim p_g}[D(\tilde{x})] - \mathbb{E}_{x\sim p_r}[D(x)] + \lambda\, \mathbb{E}_{\hat{x}\sim p_{\hat{x}}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right], \tag{8}$$

where λ controls the penalty. The penalty is evaluated at points interpolated between real and generated samples:

$$\hat{x} = \alpha x_r + (1 - \alpha) x_g, \qquad \alpha \sim U(0, 1). \tag{9}$$

The gradient penalty encourages the critic to have unit gradient norm and provides a more stable training signal than weight clipping. The generator is trained to minimize the negative critic score,

$$\mathcal{L}_G = -\mathbb{E}_{\tilde{x}\sim p_g}[D(\tilde{x})], \tag{10}$$

so that generated samples receive higher scores from the critic. Unlike weight clipping, the penalty does not restrict the critic to a narrow parameter range; it directly regularizes the gradient along lines between real and generated samples [35].

3.3 Proposed Model

The baseline PassGAN uses a WGAN-GP critic and a character-level generator with one-dimensional convolutions. The generator maps a noise vector to a sequence of characters, while the discriminator assigns a score to a password; both networks are built from residual blocks. The overall architecture is shown in Figure 1. We keep the adversarial framework and modify the generator in two ways:

1. an additional linear block is added to the generator;
2. batch normalization is inserted before each ReLU inside the residual blocks.

The modified residual block is shown in Figure 3. These changes are intended to improve gradient flow and stabilize the representation learned by the generator without changing the adversarial objective.

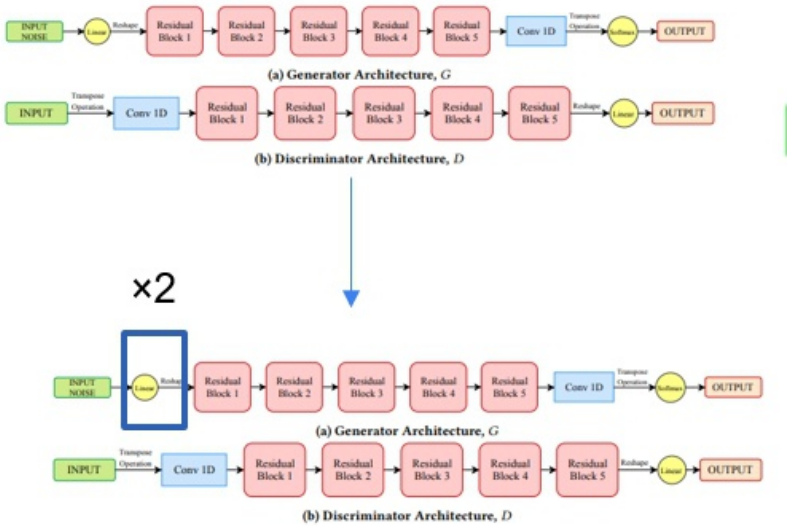
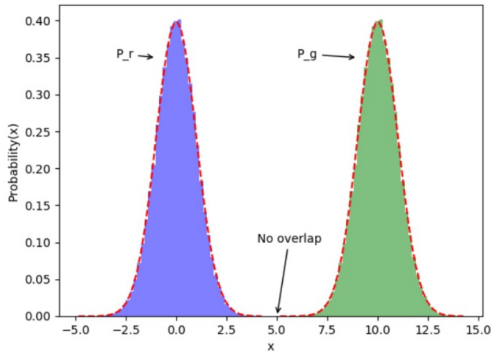
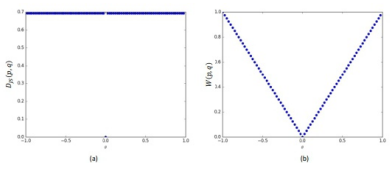


Figure 1: PassGAN generator and discriminator architecture. The generator maps a noise vector to a password; the discriminator scores real and generated passwords. Our modification adds a linear block and batch normalization to the generator.



(a) JS divergence



(b) Wasserstein distance

Figure 2: Comparison of JS divergence and Wasserstein distance. When the distributions do not overlap, JS divergence saturates, whereas Wasserstein distance still provides a useful gradient.

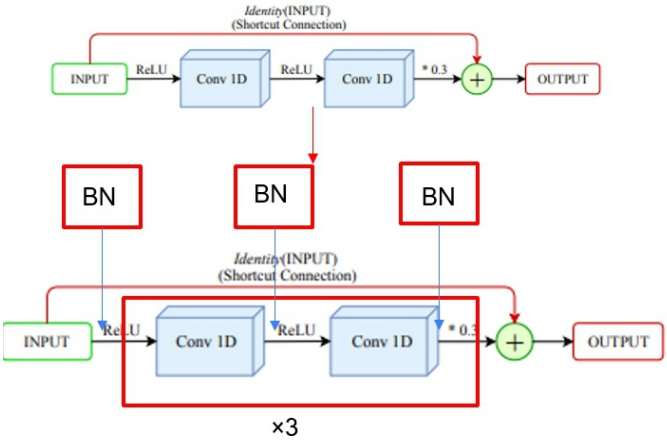


Figure 3: Modified residual block used in the generator. Batch normalization is inserted before each ReLU, and an additional linear block is added to the generator.

3.4 Data and Evaluation

We use the RockYou password corpus, a widely used collection of leaked passwords, for training and evaluation. The model is trained in Google Colab with the WGAN-GP objective in (8). Training progress is monitored at 100 and 10,000 epochs. Performance is measured by the hit rate on a held-out test set: for a candidate list of a given size, the hit rate is the fraction of test passwords matched by at least one candidate. We also report the number of guesses required for the original PassGAN to outperform each baseline.

4 Results

4.1 Training Behavior

In our runs, the generator and discriminator losses reached low values around epochs 20–30 and again around epochs 1,000–2,000. This pattern motivates early stopping: continuing training after these regions did not improve the generated password set and increased the risk of instability. Early stopping was therefore used as a practical training strategy.

We monitored training at 100 and 10,000 epochs. At 100 epochs, the losses decreased quickly and then flattened; at 10,000 epochs, the curves showed a similar pattern but with more fluctuation. The generated samples were qualitatively inspected throughout training: early samples contained many malformed strings, while later samples increasingly resembled common password patterns. The word cloud in Figure 5 summarizes the dominant patterns in the generated set.

4.2 Comparison with Baselines

Table 1 compares the proposed model with Markov models, HashCat, FLA, and PassGAN on the RockYou test set. For each baseline, the table reports the number of unique passwords, the matches obtained by the baseline, the number of guesses required for PassGAN to outperform the baseline, the PassGAN matches, and the matches obtained by our model.

The modified model obtains higher hit rates than PassGAN for all three baselines. For example, against FLA, PassGAN matches 653,978 passwords (33.06%), while the modified model matches 869,596 (43.96%).

Table 1: Comparison on the RockYou test set. Hit rates are shown in parentheses.

Approach	Unique passwords	Matches	Guesses needed for PassGAN to outperform	PassGAN matches	Our GAN matches
Markov model (3-gram)	4.9×10^8	532,961 (26.93%)	2.47×10^9	532,962 (26.93%)	678,922 (34.76%)
HashCat gen2	10^9	597,899 (30.22%)	4.8×10^9	625,245 (31.60%)	832,802 (42.09%)
FLA	7.4×10^8	652,585 (32.99%)	6×10^9	653,978 (33.06%)	869,596 (43.96%)

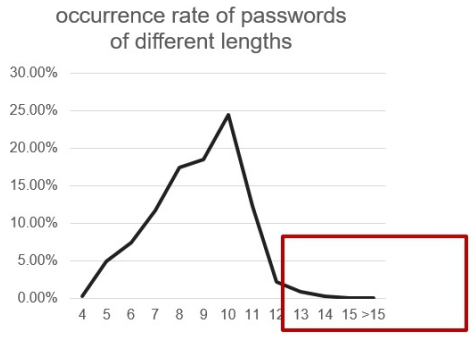
The same trend holds for HashCat gen2 (625,245 versus 832,802) and the 3-gram Markov model (532,962 versus 678,922). The improvement is therefore consistent across baselines with different candidate-generation strategies. These results indicate that the architectural changes improve the generator's ability to cover the empirical password distribution rather than exploiting a particular baseline.

4.3 Length Analysis

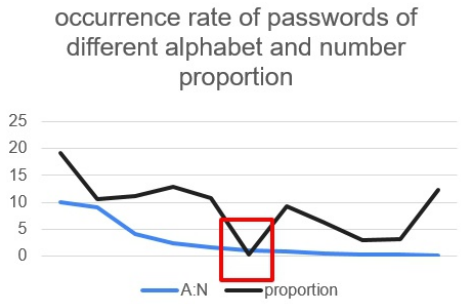
Figure 4(a) shows the distribution of password lengths in the generated/evaluated data. Passwords of length 4–12 dominate the distribution, with a peak around 10 characters. Passwords longer than 15 characters are rare, and the model struggles to guess them correctly. This is consistent with the intuition that longer passwords occupy a much larger search space and are less represented in the training corpus.

4.4 Character Composition

Figure 4(b) analyzes the relationship between the letter-to-digit ratio and guessing success. The hit rate reaches a minimum when the proportion of letters and numbers is close to 1:1. Passwords near this ratio appear to be harder for the model to guess than passwords dominated by one character class. Adding special symbols and avoiding a rigid letter–digit pattern therefore increases the effective search space.



(a) Password length



(b) Letter-to-digit ratio

Figure 4: Password length and character composition. (a) Passwords longer than 15 characters are rare. (b) The hit rate reaches a minimum around a 1:1 letter-to-digit ratio.

The word cloud provides a qualitative view of the learned distribution. Common names, affectionate phrases, and simple variants dominate the generated samples, which explains why dictionary-based and rule-based methods remain effective. It also shows that the model has learned structure beyond individual characters: it reproduces recurring phrases and naming patterns rather than generating uniformly random strings.



Figure 5: Word cloud of generated passwords. Common names, phrases, and simple variants dominate the distribution.

5 Discussion

The results support the view that GAN-based guessing is a useful tool for estimating password strength. The modified generator improves the hit rate over PassGAN, and the analysis of length and character composition provides actionable guidance. The word cloud in Figure 5 also illustrates why dictionary-based and rule-based methods remain effective: common names, phrases, and predictable variants dominate the generated distribution.

Several limitations should be noted. First, the evaluation uses a single corpus (RockYou); cross-dataset generalization remains to be tested. Second, the hit-rate metric depends on the candidate-list size and the train/test split. Third, the architectural changes were evaluated as a whole; an ablation study would be needed to isolate the contribution of the linear block and batch normalization. Finally, research on password guessing has dual-use implications. The goal of this work is defensive: better strength estimation can inform password policies, user education, and authentication design.

The security implications are twofold. On the defensive side, a realistic guessing model provides a more reliable estimate of password strength than composition rules alone, and it can be used to audit password policies before deployment. On the offensive side, the same model could be used to generate candidate lists for attacks. This asymmetry is inherent to password-guessing research. We therefore report aggregate results and focus on evaluation and user recommendations rather than releasing a production attack tool.

5.1 Future Work

Several directions remain open. First, the model should be evaluated on additional corpora and on passwords from different languages and services. Second, an ablation study should isolate the effects of the additional linear block and batch normalization. Third, the model could be combined with rule-based or Markov-based candidate generation to exploit complementary strengths. Finally, defenses such as rate limiting, breached-password detection, and phishing-resistant authentication should be evaluated against learned guessing models.

5.2 Practical Recommendations

The findings suggest four practical recommendations:

1. use passwords with at least 15 characters;
2. mix letters, digits, and special symbols rather than relying on a single character class;
3. avoid reusing passwords across services;
4. enable two-factor or multi-factor authentication where available.

These measures increase the search space and reduce the value of a single leaked credential.

6 Conclusion

This paper presented a GAN-based password-guessing model built on WGAN-GP and a modified PassGAN generator. The model adds a linear block and batch normalization to the residual generator, and it outperforms the original PassGAN on the RockYou test set across three baselines. The analysis of password length and character composition leads to concrete recommendations for users. Future work should evaluate cross-dataset generalization, perform ablation studies, and explore defenses that remain robust against learned guessing models.

References

[1] B. Hitaj, P. Gasti, G. Ateniese, and F. Perez-Cruz, "PassGAN: A deep learning approach for password guessing," in *Proc. ACNS*, 2019, pp. 217–237.

[2] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in *Proc. IEEE Symposium on Security and Privacy*, 2012, pp. 553–567.

[3] A. Narayanan and V. Shmatikov, "Fast dictionary attacks on passwords using time–space tradeoff," in *Proc. ACM CCS*, 2005, pp. 364–372.

[4] S. Pearman, J. Thomas, P. E. Naeini, H. Habib, L. Bauer, N. Christin, L. F. Cranor, S. Egelman, and A. Forget, "Let's go in for a closer look: Observing passwords in their natural habitat," in *Proc. ACM CCS*, 2017, pp. 295–310.

[5] DeepMind, "AlphaGo," 2018. [Online]. Available: https://deepmind.com/research/alphago/

[6] M. Hutson, "Artificial intelligence just made guessing your password a whole lot easier," *Science*, 2017.

[7] I. Thomson, "AI slurps, learns millions of passwords to work out which ones you may use next," *The Register*, 2017.

[8] A. Manning, "Researchers show how A.I. is the end of passwords as we know them," *Inverse*, 2017.

[9] M. Mimoso, "Deep-learning PassGAN tool improves password guessing," *Threatpost*, 2017.

[10] J. Vijayan, "PassGAN: Password cracking using machine learning," *Dark Reading*, 2017.

[11] J. Condliffe, "A pair of AIs have become very good at guessing your passwords," *Technology Review*, 2017.

[12] M. Dirjish, "Artificial intelligence, slayer of passwords," *Sensors Online*, 2017.

[13] M. Cobb, "How machine learning-powered password guessing impacts security," *TechTarget*, 2017.

[14] J. W. Goerlich, "PassGAN for AI password guessing," *Stuck in Traffic*, 2017.

[15] E. Intini, "L'intelligenza artificiale indovina le password," *Focus IT*, 2017.

[16] A. Janofsky, "AI could make cyberattacks more dangerous, harder to detect," *The Wall Street Journal*, 2018.

[17] I. Madan, "Up to speed on AI and deep learning: October update," *Hackernoon*, 2017.

[18] Stevens Institute of Technology, "These powerful new password crackers challenge our notions of secure," 2017.

[19] K. J. Higgins, "The coolest hacks of 2017," *Dark Reading*, 2017.

[20] UC Berkeley, "Arxiv summaries week 09/11: PassGAN: A deep learning approach for password guessing," CS 294-131: Special Topics in Deep Learning, 2017.

[21] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," Technical Report, 2016.

[22] N. Provos and D. Mazières, "Bcrypt algorithm," in *Proc. USENIX*, 1999.

[23] S. Sun, J. Fan, J. Qu, and P. Lu, "A survey of generative adversarial networks," *Computer Engineering and Applications*, no. 18, pp. 90–103, 2022 (in Chinese).

[24] G.-J. Qi, "Loss-sensitive generative adversarial networks on Lipschitz densities," *International Journal of Computer Vision*, vol. 128, no. 5, pp. 1118–1140, 2020.

[25] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. ICML*, 2017, pp. 214–223.

[26] Wikipedia contributors, "John the Ripper," *Wikipedia, The Free Encyclopedia*, 2022.

[27] W. Tansey, "Improved models for password guessing," University of Texas, Tech. Rep., 2011.

[28] J. Ma, W. Yang, M. Luo, and N. Li, "A study of probabilistic password models," in *Proc. IEEE Symposium on Security and Privacy*, 2014, pp. 689–704.

[29] M. Dürmuth, F. Angelstorf, C. Castelluccia, D. Perito, and A. Chaabane, "OMEN: Faster password guessing using an ordered Markov enumerator," in *Proc. ESSoS*, 2015, pp. 119–132.

[30] W. Melicher, B. Ur, S. M. Segreti, S. Komanduri, L. Bauer, N. Christin, and L. F. Cranor, "Fast, lean, and accurate: Modeling password guessability using neural networks," in *Proc. USENIX Security*, 2016, pp. 175–191.

[31] L. Xu, C. Ge, W. Qiu, and Z. Huang, "Password guessing based on LSTM recurrent neural networks," in *Proc. IEEE CSE/EUC*, 2017, pp. 785–788.

[32] Z. Xia and P. Yi, "Multi-source deep learning model for password guessing," *Communications Technology*, 2019.

[33] T. Zhou, "Research on password guessing based on deep learning," M.S. thesis, South China University of Technology, Guangzhou, China, 2022 (in Chinese).

[34] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Proc. NeurIPS*, 2014.

[35] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of Wasserstein GANs," in *Proc. NeurIPS*, 2017.